REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Isidora Sofía Villegas Hernández

8 de abril de 2026

18:24

Resumen

En el presente informe se realiza un análisis experimental de algoritmos de ordenamiento (`MergeSort`[1], `QuickSort`[2], `StdSort`) y multiplicación de matrices (`Naive`, `Strassen`[3]), con el objetivo de evaluar su rendimiento en la práctica y comparar los resultados obtenidos con las predicciones teóricas basadas en la notación asintótica. Se implementaron diversos algoritmos tanto para algoritmos de orden como para multiplicación de matrices, se ejecutaron y se recopilaron datos sobre su tiempo de ejecución y uso de recursos. Este análisis permite identificar las diferencias entre el comportamiento teórico y real de los algoritmos, así como comprender mejor las implicaciones prácticas de sus complejidades computacionales.

1. Entorno experimental

Los experimentos fueron realizados en una máquina virtual con las siguientes características:

- **Hardware:** (máquina física)
  - Procesador:
  - Memoria Ram:
  - GPU: NVIDIA GeForce RTX 3060ti
  - Almacenamiento:
- **Detalles Máquina Virtual:**
  - Sistema Operativo: Ubuntu 24.04.1 LTS

- Compilador: g++ 13.3.0
- Recursos Asignados a Máquina Virtual:

Mediciones

1. **Tiempo de ejecución:** Se utilizó la biblioteca <chrono>de C++ para medir el tiempo transcurrido en ms durante la ejecución de cada algoritmo individualmente. Esto excluyendo operaciones de entrada/salida y compilación.

2. **Consumo de memoria:** Se empleó getrusage() para medir el consumo máximo residente (RSS) durante la ejecución de cada algoritmo. En el caso particular de los algoritmos de ordenamiento, se midió en dos ejecuciones, inicialmente solo medía el valor máximo RSS alcanzado por el proceso completo, no necesariamente la memoria extra usada por el algoritmo, por lo que se decidió medir el consumo de memoria antes y después de la ejecución del algoritmo, y restar ambos valores para obtener una estimación más precisa del consumo de memoria adicional.

3. **Compilación:** Cada medición se estandarizó con la utilización de makefiles para compilar los algoritmos con las mismas opciones de optimización (-O3) y sin incluir información de depuración (-g). Esto garantiza que las comparaciones entre algoritmos sean justas y consistentes.

4. **Casos de prueba:** Se utilizaron sólo los datasets generados por los scripts correspondientes a cada sección.

Datasets

Los datasets utilizados para los experimentos fueron generados utilizando scripts específicos para cada sección. Estos scripts se encargaron de crear conjuntos de datos con dos tipos de dominios: `D1`: {0,1}, `D7`: {0,1,2,3,4,5,6,7,8,9} en donde para algoritmos de ordenamiento la estructura de pruebas son arreglos de tipo `ascendente`, `descendente` y `aleatorio` dentro de los rangos de valores de 10^3 hasta 10^7 y para multiplicación de matrices de los tipos `dispersa`, `diagonal` y `densa` dentro de los rangos de valores de 2^2 hasta 2^{10}. Estos datasets proporcionaron una base sólida para evaluar el rendimiento de los algoritmos en diferentes escenarios y condiciones, permitiendo así obtener conclusiones más robustas sobre su comportamiento.

2. Resultados y Análisis

Algoritmos de ordenamiento

Los algoritmos de ordenamiento muestran patrones de rendimiento diferentes a las expectativas teóricas, revelando la complejidad de implementar algoritmos eficientes en entornos reales.

- **Quicksort:** Aunque teóricamente tiene un tiempo de ejecución de $O(n \log n)$ en el mejor y promedio caso, en la práctica, su rendimiento puede verse afectado por la elección del pivote. En casos

donde el pivote no se selecciona adecuadamente (por ejemplo, si el arreglo ya está ordenado), el tiempo de ejecución puede degradarse a $O(n^2)$. Dado que la implementación utilizada selecciona un pivote aleatorio pero ciertos casos pueden no ser tan aleatorios, se observa que el rendimiento de Quicksort puede variar significativamente, especialmente con entradas grandes o patrones reiterativos de datos.

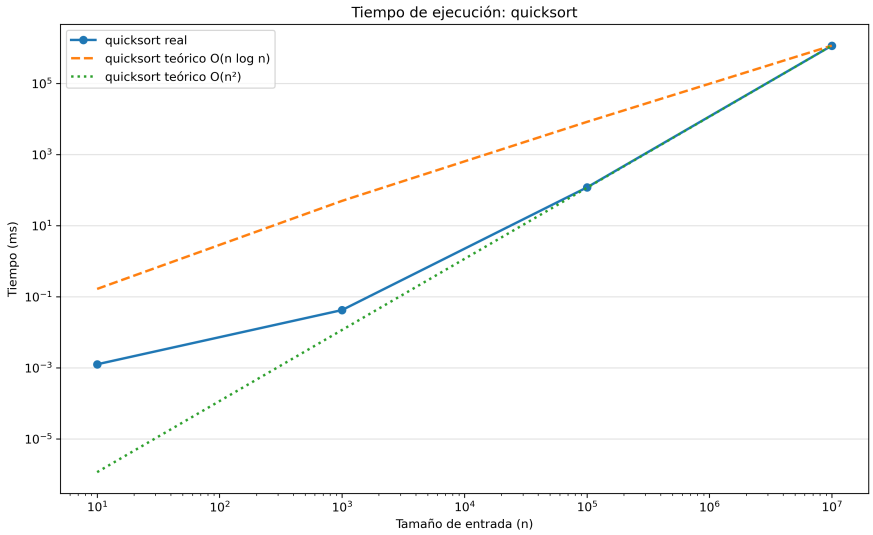

Figura 1: Rendimiento de Quicksort comparado a rendimientos teóricos.

- **Mergesort:** En la ejecución de este algoritmo se dieron resultados consistentes con el tiempo de ejecución teórico de $O(n \log n)$, independientemente de la naturaleza de los datos de entrada. Esto se debe a que Mergesort siempre divide el arreglo en partes iguales y combina los resultados de manera eficiente, lo que garantiza un rendimiento estable y de esa manera se cataloga como el algoritmo más robustos frente a variaciones en las características de los datos.ç

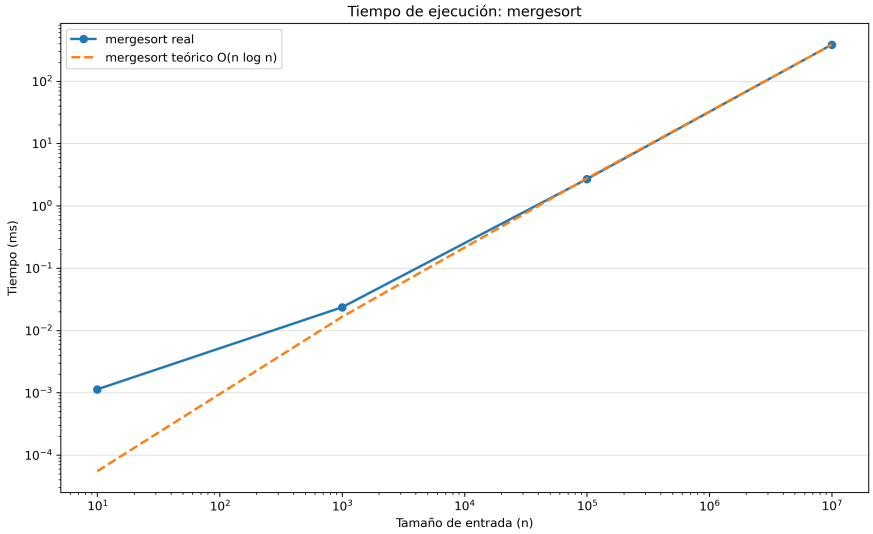

Figura 2: Rendimiento de Mergesort comparado a rendimientos teóricos.

3. Conclusiones

La extensión máxima para esta sección es un párrafo breve (máximo 1/4 de página).

Escriba una conclusión breve que interprete los resultados más importantes. Cada afirmación debe estar respaldada por los datos presentados.

Referencias

[1] Geeks for Geeks. *Merge Sort*. https://www.geeksforgeeks.org/dsa/merge-sort/. 2025.

[2] Geeks for Geeks. *Quick Sort*. https://www.geeksforgeeks.org/cpp/cpp-program-for-quicksort/. 2025.

[3] Geeks for Geeks. *Strassen's Matrix Multiplication*. https://www.geeksforgeeks.org/dsa/strassens-matrix-multiplication/. 2025.